



Gabriel dos Santos Silva Coelho

RESENHA

Artigo 8 - Hexagonal Architecture

O artigo "Hexagonal Architecture", lançado pelo Alistair Cockburn lá em 2005, é um verdadeiro divisor de águas na engenharia de software. O Cockburn, que é um dos "pais" do Manifesto Ágil e referência total em Orientação a Objetos, apresenta nesse texto o que muita gente também conhece como "Ports and Adapters" (Portas e Adaptadores).

A grande sacada aqui é simples, mas poderosa: o objetivo é blindar o núcleo da aplicação. A ideia é que o sistema funcione do mesmo jeito, não importa quem esteja chamando, seja um usuário na tela, um script automatizado ou um processo em lote. Com essa estrutura, você consegue desenvolver e testar toda a lógica de negócio de forma isolada, sem ficar dependente de banco de dados ou de ferramentas externas que só rodam em tempo de execução.

Na prática, a Arquitetura Hexagonal divide o sistema em dois lados: o interior, onde mora a lógica de negócio (o que realmente importa), e o exterior, onde ficam todos os agentes externos. A ponte entre esses dois mundos é feita por portas, que são apenas interfaces que definem como a conversa deve ser, e adaptadores, que são as implementações reais que conectam cada tecnologia a uma porta.

E um detalhe curioso: o nome "hexágono" não tem nada a ver com matemática ou números. O Cockburn escolheu essa forma só para ter mais espaço visual para desenhar várias portas e adaptadores. Isso ajuda a gente a sair daquela cilada de desenhar tudo em camadas empilhadas, o que acabava gerando acoplamentos desnecessários. A ideia é manter a simetria: existe uma aplicação no centro conversando com vários elementos de

fora, e todos esses elementos externos são tratados da mesma forma pelo núcleo do sistema.

A maior vitória deste artigo foi dar nome a algo que muito desenvolvedor experiente já sentia na pele, mas não sabia explicar. O padrão nasceu da raiva de ver regra de negócio misturada com código de tela, principalmente naqueles sistemas Java antigos onde tudo era dividido em camadas rígidas. Ao criar o nome e a metáfora visual, o Cockburn deu um vocabulário novo para discutirmos a arquitetura.

A grande vantagem é que o sistema fica "solto" (desacoplado). Você testa a aplicação sem precisar de banco de dados ou de uma tela pronta, o que evita que você fique preso a uma tecnologia específica. Isso obriga o dev a pensar de verdade nas dependências, acabando com aquela hierarquia chata onde a "camada de cima" manda em tudo.

Mas o texto original tem seus buracos. Ele é bem curto e não traz exemplos de código robustos para projetos gigantes. É por isso que cada um interpreta de um jeito e muita gente confunde com *Clean Architecture* ou *DDD*. Detalhes práticos, como "onde coloco tal pasta" ou "como gerencio o ciclo de vida dos componentes", ficaram no ar.

Além disso, o padrão exige uma disciplina que custa caro quando o prazo está apertado. Criar adaptador para tudo dá trabalho no começo. E para sistemas modernos, tipo arquiteturas orientadas a eventos que o artigo de 2005 nem imaginava, a distinção entre quem "manda" e quem "obedece" na conversa pode ficar meio confusa.